

## PRACTICAL - 12

Name: S Rikhil

Roll No: 2520030480

Sec: 8

### PART 1: PRODUCER-CONSUMER PROBLEM

#### Aim

To implement the Producer-Consumer problem using counting semaphores and POSIX threads, and evaluate synchronization correctness and system throughput under different buffer sizes.

#### Theory

The Producer-Consumer problem involves a producer adding items to a shared buffer and a consumer removing them. Counting semaphores ensure that the producer does not add to a full buffer and the consumer does not remove from an empty buffer.

#### C Code

- include <stdio.h>
- include <pthread.h>
- include <semaphore.h>

- define BUFFER\_SIZE 5
- define ITEMS 20

```
int buffer[BUFFER_SIZE];  
int in = 0, out = 0;
```

```
sem_t empty, full;  
pthread_mutex_t mutex;
```

```
void *producer(void *arg)  
{  
    for (int i = 1; i <= ITEMS; i++)  
    {  
        sem_wait(&empty);  
  
        pthread_mutex_lock(&mutex);  
  
        buffer[in] = i; printf("Produced:  
%d\n", i); in = (in + 1) %  
        BUFFER_SIZE;
```

```

        pthread_mutex_unlock(&mutex);

        sem_post(&full);
    }

    return NULL;
}

void *consumer(void *arg)
{
    for (int i = 1; i <= ITEMS; i++)
    {
        sem_wait(&full);

        pthread_mutex_lock(&mutex);

        int item = buffer[out];
        printf("Consumed: %d\n", item); out
        = (out + 1) % BUFFER_SIZE;

        pthread_mutex_unlock(&mutex);

        sem_post(&empty);
    }

    return NULL;
}

int main()
{
    pthread_t producer_thread, consumer_thread;

    sem_init(&empty, 0, BUFFER_SIZE);
    sem_init(&full, 0, 0);
    pthread_mutex_init(&mutex, NULL);

    pthread_create(&producer_thread, NULL, producer, NULL);
    pthread_create(&consumer_thread, NULL, consumer, NULL);

    pthread_join(producer_thread, NULL);
    pthread_join(consumer_thread, NULL);

    sem_destroy(&empty);
    sem_destroy(&full);
}

```

```
pthread_mutex_destroy(&mutex);
```

```
    return 0;  
}
```

OUTPUT –

```
teja@Hasini:~$ ./pc_demo  
Produced: 1  
Produced: 2  
Produced: 3  
Produced: 4  
Produced: 5  
Consumed: 1  
Consumed: 2  
Consumed: 3  
Consumed: 4  
Consumed: 5  
Produced: 6  
Produced: 7  
Produced: 8  
Produced: 9  
Produced: 10  
Consumed: 6  
Consumed: 7  
Consumed: 8  
Consumed: 9  
Consumed: 10  
Produced: 11  
Produced: 12  
Produced: 13  
Produced: 14  
Produced: 15  
Consumed: 11  
Consumed: 12  
Consumed: 13  
Consumed: 14  
Consumed: 15  
Produced: 16  
Produced: 17  
Produced: 18  
Produced: 19  
Produced: 20  
Consumed: 16
```

Compile

```
gcc producer_consumer.c -o pc -pthread
```

Run

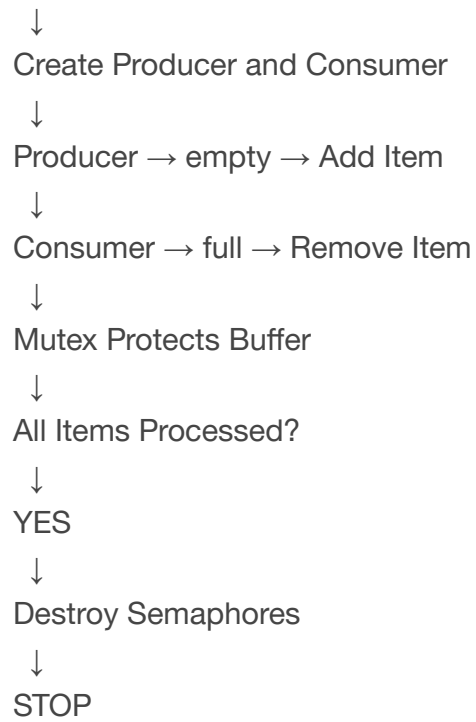
```
./pc
```

Flow Chart

START

↓

Initialize Buffer and Semaphores



### Buffer Size Evaluation

| Buffer Size | Synchronization | Throughput        |
|-------------|-----------------|-------------------|
| 2           | Correct         | Lower             |
| 5           | Correct         | Good              |
| 10          | Correct         | Higher            |
| 20          | Correct         | Depends on system |

Larger buffers can reduce waiting between producer and consumer, but the actual throughput depends on CPU, scheduling, and workload.

### OUTPUT:

#### Result

The Producer-Consumer problem was successfully implemented using POSIX

threads, counting semaphores, and a mutex. Synchronization was maintained correctly and buffer size affected throughput.

## PART 2: DEADLOCK AND PREVENTION USING RESOURCE ORDERING

### Aim

To design a deadlock scenario involving multiple threads competing for shared resources, demonstrate deadlock occurrence, identify the four necessary conditions for deadlock, and prevent deadlock using resource ordering.

### Theory

A deadlock occurs when threads wait indefinitely for resources held by each other.

The four necessary conditions are:

1. Mutual Exclusion – A resource can be used by only one thread at a time.
2. Hold and Wait – A thread holds one resource while waiting for another.
3. No Preemption – A resource cannot be forcibly taken from a thread.
4. Circular Wait – Threads form a circular chain of waiting.

### Deadlock Code

- include <stdio.h>
- include <pthread.h>
- include <unistd.h>

```
pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;  
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void *thread1(void *arg)  
{  
    pthread_mutex_lock(&resource1); printf("Thread 1  
    locked Resource 1\n");  
  
    sleep(1);  
  
    pthread_mutex_lock(&resource2); printf("Thread 1  
    locked Resource 2\n");  
  
    pthread_mutex_unlock(&resource2);  
    pthread_mutex_unlock(&resource1);  
  
    return NULL;  
}
```

```

void *thread2(void *arg)
{
    pthread_mutex_lock(&resource2); printf("Thread 2
    locked Resource 2\n");

    sleep(1);

    pthread_mutex_lock(&resource1); printf("Thread 2
    locked Resource 1\n");

    pthread_mutex_unlock(&resource1);
    pthread_mutex_unlock(&resource2);

    return NULL;
}

int main()
{
    pthread_t t1, t2;

    pthread_create(&t1, NULL, thread1, NULL); pthread_create(&t2,
    NULL, thread2, NULL);

    pthread_join(t1, NULL); pthread_join(t2,
    NULL);

    return 0;
}

```

## Compile

gcc deadlock.c -o deadlock -pthread

## OUTPUT:

```

Consumed: 20
teja@Hasini:~$ nano deadlock_demo.c
teja@Hasini:~$ gcc deadlock_demo.c -o deadlock_demo -pthread
teja@Hasini:~$ ./deadlock_demo
Thread 2 locked Resource 2
Thread 1 locked Resource 1

```

## Run

./deadlock

The program may remain stuck because each thread waits for a resource held by the other.

## PART 3: DEADLOCK PREVENTION USING RESOURCE ORDERING

### Aim

To prevent deadlock by forcing all threads to acquire shared resources in the same order.



## C Code

- include <stdio.h>
- include <pthread.h>
- include <unistd.h>

```
pthread_mutex_t resource1 = PTHREAD_MUTEX_INITIALIZER;  
pthread_mutex_t resource2 = PTHREAD_MUTEX_INITIALIZER;
```

```
void *thread1(void *arg)  
{  
    pthread_mutex_lock(&resource1); printf("Thread 1  
    locked Resource 1\n");  
  
    sleep(1);  
  
    pthread_mutex_lock(&resource2); printf("Thread 1  
    locked Resource 2\n");  
  
    pthread_mutex_unlock(&resource2);  
    pthread_mutex_unlock(&resource1);  
  
    return NULL;  
}
```

```
void *thread2(void *arg)  
{  
    /* Same order: Resource 1 first, Resource 2 second */  
    pthread_mutex_lock(&resource1);  
    printf("Thread 2 locked Resource 1\n"); sleep(1);  
  
    pthread_mutex_lock(&resource2); printf("Thread 2  
    locked Resource 2\n");  
  
    pthread_mutex_unlock(&resource2);  
    pthread_mutex_unlock(&resource1);  
  
    return NULL;  
}
```

```
int main()  
{  
    pthread_t t1, t2;  
  
    pthread_create(&t1, NULL, thread1, NULL); pthread_create(&t2,
```

```
NULL, thread2, NULL);
```

```
pthread_join(t1, NULL); pthread_join(t2,
NULL);

printf("No deadlock occurred.\n");

return 0;
}
```

## Compile

```
gcc prevention.c -o prevention -pthread
```

## Run

```
./prevention
```

## Expected Output

```
Thread 1 locked Resource 1
Thread 1 locked Resource 2
Thread 2 locked Resource 1
Thread 2 locked Resource 2 No
deadlock occurred.
```

## Flow Chart

START



Create Two Resources



Create Multiple Threads



Acquire Resources



Different Order?



YES → DEADLOCK



Apply Resource Ordering



Resource 1 → Resource 2



Threads Complete



STOP

## Observation

In the deadlock program, each thread locks one resource and waits for the

other, creating a circular wait. By forcing all threads to acquire resources in the same order, circular wait is eliminated.

#### OUTPUT:

```
teja@Hasini:~$ nano no_deadlock.c
teja@Hasini:~$ gcc no_deadlock.c -o no_deadlock -pthread
teja@Hasini:~$ ./no_deadlock
Thread 1 locked Resource 1
Thread 1 locked Resource 2
Thread 2 locked Resource 1
Thread 2 locked Resource 2
No deadlock occurred.
teja@Hasini:~$ |
```

#### Result

Deadlock was successfully demonstrated and prevented using resource ordering. The prevention strategy eliminates the circular-wait condition and allows all threads to complete successfully.